

Fiche 314 — Graphiques : haut niveau, bas niveau, `par()` et pilotes de périphérique

Matière	Code · Langage R
Cours source	R Core Team, <i>An Introduction to R</i> 4.6.1 — chapitre 12 « Graphical procedures » (§12.1 commandes de haut niveau, §12.2 commandes de bas niveau, §12.3 interaction, §12.4 paramètres graphiques, §12.5 liste des paramètres, §12.6 pilotes de périphérique, §12.7 graphiques dynamiques)
Difficulté	Intermédiaire — long chapitre, mais organisé autour de trois idées seulement
Temps d'étude estimé	1 h 45
Prérequis	Fiches 304, 306, 311, 313 (facteurs, data frames, dispatch S3, formules)
Concepts clés	les trois groupes de commandes, <code>plot()</code> générique , <code>pairs()</code> , <code>coplot()</code> , <code>panel=</code> , <code>type=</code> , <code>add=TRUE</code> , <code>axes=FALSE</code> , <code>points</code> , <code>lines</code> , <code>text</code> , <code>abline</code> , <code>polygon</code> , <code>legend</code> , <code>title</code> , <code>axis</code> , coordonnées utilisateur , <code>expression()</code> et <code>plotmath</code> , polices de Hershey, <code>locator()</code> , <code>identify()</code> , <code>par()</code> global , <code>oldpar</code> , <code>mfrow</code> / <code>mfcoll</code> , <code>mar</code> / <code>mai</code> , <code>oma</code> / <code>omi</code> , pilotes, <code>dev.off()</code> , <code>graphics.off()</code>
À retenir en priorité	Haut niveau efface, bas niveau ajoute · <code>plot()</code> est générique · <code>par()</code> est global, même appelé dans une fonction · l'idiome <code>oldpar</code> · <code>dev.off()</code> .

Vue d'ensemble

TROIS GROUPES DE COMMANDES

HAUT NIVEAU creent un plot COMPLET -- « ALWAYS START A NEW PLOT, ERASING THE CURRENT PLOT IF NECESSARY »

BAS NIVEAU AJOUTENT a un trace existant : points, lignes, texte

INTERACTIVES `locator()` `identify()` -- a la souris

+ UNE LISTE DE PARAMETRES GRAPHIQUES, manipulables par `par()`

`plot()` EST GENERIQUE « the type of plot produced is DEPENDENT ON THE TYPE OR CLASS OF THE FIRST ARGUMENT »

```
plot(x, y) nuage      plot(f)      diagramme en barres d'un facteur
plot(x)      serie/index  plot(f, y) boites a moustaches par niveau
plot(df)      distributions  plot(y ~ expr) y contre chaque objet
```

```
MULTIVARIE  pairs(X)      matrice de nuages deux a deux
            coplot(a ~ b | c) et coplot(a ~ b | c + d)
            argument panel= pour changer le trace de chaque case
```

ARGUMENTS DE HAUT NIVEAU

```
add=TRUE axes=FALSE log="x"/"y"/"xy" xlab= ylab= main= sub=
type= : p points l lignes b les deux o superposes
      h batons s/S escaliers n RIEN (mais les axes sont traces)
```

```
par() !! « calls to par() ALWAYS AFFECT THE GLOBAL VALUES ... EVEN WHEN
       par() IS CALLED FROM WITHIN A FUNCTION »
oldpar <- par(no.readonly = TRUE) ... par(oldpar)
```

```
PERIPHERIQUES  X11() windows() quartz() postscript() pdf() png() jpeg()
               dev.new() dev.list() dev.set() dev.off(k) graphics.off()
```

Le problème posé. « *Graphical facilities are an important and extremely versatile component of the R environment. It is possible to use the facilities to display a wide variety of statistical graphs and also to build entirely new types of graph.* » (§12)

INTUITION — L'ORGANISATION DU CHAPITRE TIENT EN UNE PHRASE.

« *Plotting commands are divided into three basic groups : high-level ... create a new plot ; low-level ... add more information to an existing plot ; interactive ...* » Tout le reste est du catalogue. **Savoir dans quel groupe on est** suffit à prévoir si la commande va effacer ou compléter.

Le démarrage (§12). « *at startup time R initiates a graphics device driver which opens a special graphics window ... it may be useful to know that the command used is `X11()` under UNIX, `windows()` under Windows and `quartz()` under macOS. A new device can always be opened by `dev.new()`.* »

Le périmètre du chapitre (§12). « *This manual only describes what are known as "base" graphics. A separate graphics sub-system in package `grid` coexists with base — it is more powerful but harder to use. There is a recommended package `Lattice` which builds on `grid` and provides ways to produce multi-panel plots akin to those in the Trellis system in S.* »

● Concept 1 — Les commandes de haut niveau

DÉFINITION (§12.1).

« High-level plotting functions are designed to **generate a complete plot of the data passed as arguments**. Where appropriate, **axes, labels and titles are automatically generated** (unless you request otherwise). **High-level plotting commands always start a new plot, erasing the current plot if necessary.** »

1.1 `plot()` — une fonction, sept comportements

Règle (§12.1.1). « *This is a generic function : the type of plot produced is dependent on the type or class of the first argument.* »

C'est le dispatch S3 de la fiche 311, ici à pleine échelle :

Appel	Ce que R produit
<code>plot(x, y)</code>	« If <code>x</code> and <code>y</code> are vectors ... a scatterplot of <code>y</code> against <code>x</code> »
<code>plot(xy)</code>	« The same effect ... supplying one argument ... as either a list containing two elements <code>x</code> and <code>y</code> or a two-column matrix »
<code>plot(x)</code> — série temporelle	« a time-series plot »
<code>plot(x)</code> — vecteur numérique	« a plot of the values in the vector against their index »
<code>plot(x)</code> — vecteur complexe	« a plot of imaginary versus real parts »
<code>plot(f)</code> — facteur	« a bar plot of <code>f</code> »
<code>plot(f, y)</code> — facteur et numérique	« boxplots of <code>y</code> for each level of <code>f</code> »
<code>plot(df)</code> — data frame	« distributional plots of the variables »
<code>plot(~ expr)</code>	des tracés distributionnels « of a number of named objects »
<code>plot(y ~ expr)</code>	« plots <code>y</code> against every object named in <code>expr</code> »

« `expr` is **a list of object names separated by `+`** (e.g. `a + b + c`). »

EN CLAIR — DEUX ANNEXES A ENFIN EXPLIQUÉES.

La fiche 300 relevait deux surprises : `plot(Expt, Speed)` produisant **des boîtes à moustaches**, et `plot(z, type="l")` traçant **imaginaire contre réel**. Les lignes 7 et 5 du tableau les expliquent. Et `plot(fr)` sur une "table" (fiche 305) relève du même mécanisme : « because it "sees" that `fr` is of class "table" ».

1.2 Les données multivariées

DÉFINITION (§12.1.2).

« If `x` is a numeric matrix or data frame, `pairs(x)` produces a pairwise scatterplot matrix of the variables ... every column of `x` is plotted against every other column and the resulting $n(n - 1)$ plots are arranged in a matrix with plot scales constant over the rows and columns. »

DÉFINITION (§12.1.2).

« When three or four variables are involved a coplot may be more enlightening. »

```
coplot(a ~ b | c)      # a contre b, POUR chaque valeur de c
coplot(a ~ b | c + d)  # « for every JOINT conditioning interval of c and d »
```

« If `c` is a factor, this simply means that `a` is plotted against `b` for every level of `c`. When `c` is numeric, it is divided into a number of conditioning intervals and for each interval `a` is plotted against `b` for values of `c` within the interval. »

« The number and position of intervals can be controlled with `given.values=` — the function `co.intervals()` is useful for selecting intervals. »

MÉTHODE — PERSONNALISER CHAQUE CASE (§12.1.2).

« The `coplot()` and `pairs()` function both take an argument `panel=` which can be used to customize the type of plot which appears in each panel. The default is `points()` to produce a scatterplot, but by supplying some other low-level graphics function of two vectors `x` and `y` as the value of `panel=` you can produce any type of plot you wish. An example panel function useful for coplots is `panel.smooth()`. »

C'est une fonction passée en argument (fiche 309) : `panel=` reçoit un objet fonction, exactement comme `outer(x, y, f)`.

1.3 Les autres tracés de haut niveau

Fonction (§12.1.3)	Ce qu'elle produit
<code>qqnorm(x)</code>	« plots <code>x</code> against the expected Normal order scores (a normal scores plot) »
<code>qqline(x)</code>	« adds a straight line ... by drawing a line through the distribution and data quartiles »
<code>qqplot(x, y)</code>	« plots the quantiles of <code>x</code> against those of <code>y</code> »
<code>hist(x)</code>	un histogramme ; « A sensible number of classes is usually chosen »
<code>hist(x, nclass=n)</code>	« a recommendation can be given » — donc un souhait, pas un ordre
<code>hist(x, breaks=b)</code>	« the breakpoints can be specified exactly »
<code>hist(x, probability=TRUE)</code>	« the bars represent relative frequencies divided by bin width instead of counts »
<code>dotchart(x)</code>	« the y-axis gives a labelling of the data and the x-axis gives its value ... allows easy visual selection of all data entries with values lying in specified ranges »
<code>image(x, y, z)</code>	« draws a grid of rectangles using different colours to represent the value of <code>z</code> »
<code>contour(x, y, z)</code>	« draws contour lines »
<code>persp(x, y, z)</code>	« draws a 3D surface »

`nclass` est une recommandation, `breaks` une instruction. Le cours choisit ses mots : « a recommendation can be given » contre « specified **exactly** ». Un `nclass = 20` peut donner 18 ou 25 classes.

1.4 Les arguments communs

NOTATION (§12.1.4).

« There are a number of arguments which may be passed to high-level graphics functions » :

Argument	Effet
<code>add=TRUE</code>	« Forces the function to act as a low-level graphics function , superimposing the plot on the current plot (some functions only) »
<code>axes=FALSE</code>	« Suppresses generation of axes — useful for adding your own custom axes with the <code>axis()</code> function »
<code>log="x"</code> · <code>log="y"</code> · <code>log="xy"</code>	« Causes the x, y or both axes to be logarithmic. This will work for many, but not all, types of plot. »
<code>xlab=</code> · <code>ylab=</code>	« Axis labels ... usually the names of the objects used in the call »
<code>main=</code>	« Figure title, placed at the top ... in a large font »
<code>sub=</code>	« Sub-title, placed just below the x-axis ... in a smaller font »

Règle — les huit valeurs de `type=` (§12.1.4).

Valeur	Effet
<code>"p"</code>	« Plot individual points (the default) »
<code>"l"</code>	« Plot lines »
<code>"b"</code>	« points connected by lines (both) »
<code>"o"</code>	« points overlaid by lines »
<code>"h"</code>	« vertical lines from points to the zero axis (high-density) »
<code>"s"</code> / <code>"S"</code>	« Step-function plots. In the first form the top of the vertical defines the point ; in the second, the bottom. »
<code>"n"</code>	« No plotting at all. However axes are still drawn (by default) and the coordinate system is set up according to the data. Ideal for creating plots with subsequent low-level graphics functions. »

`type="n"` est le plus utile des huit : il **prépare le terrain** — cadre, échelles, axes — sans rien tracer. Tout ce qui suivra sera du bas niveau, entièrement maîtrisé.

● Concept 2 — Les commandes de bas niveau

Cadrage (§12.2). « Sometimes the high-level plotting functions **don't produce exactly the kind of plot you desire**. In this case, **low-level plotting commands can be used to add extra information (such as points, lines or text) to the current plot.** »

Fonction	Ce qu'elle ajoute (§12.2)
<code>points(x, y)</code> · <code>lines(x, y)</code>	des points ou des lignes ; « <code>plot()</code> 's <code>type=</code> argument can also be passed ... and defaults to "p" for <code>points()</code> and "l" for <code>lines()</code> »
<code>text(x, y, labels, ...)</code>	du texte aux points donnés ; « <code>labels[i]</code> is plotted at point <code>(x[i], y[i])</code> . The default is <code>1:length(x)</code> . »
<code>abline(a, b)</code>	« a line of slope <code>b</code> and intercept <code>a</code> »
<code>abline(h=y)</code> · <code>abline(v=x)</code>	des lignes horizontales / verticales
<code>abline(lm.obj)</code>	« <code>lm.obj</code> may be a list with a <code>coefficients</code> component of length 2 ... taken as an intercept and slope, in that order. »
<code>polygon(x, y, ...)</code>	« Draws a polygon defined by the ordered vertices ... and (optionally) shade it in with hatch lines, or fill it if the graphics device allows »
<code>legend(x, y, legend, ...)</code>	une légende ; voir ci-dessous
<code>title(main, sub)</code>	un titre et un sous-titre
<code>axis(side, ...)</code>	« Adds an axis ... on the side given by the first argument (1 to 4, counting clockwise from the bottom) ... Useful for adding custom axes after calling <code>plot()</code> with the <code>axes=FALSE</code> argument. »

MÉTHODE — L'IDIOME TYPE="N" + TEXT() (§12.2).

« **Note** : This function is often used in the sequence »

```
plot(x, y, type = "n"); text(x, y, names)
```

« **The graphics parameter** `type="n"` **suppresses the points but sets up the axes**, and the `text()` function supplies special characters, as specified by the character vector `names` for the points. »

On remplace les points par leurs étiquettes — un nuage où chaque observation est nommée.

Règle — les arguments de `legend()` (§12.2). « **At least one other argument** `v` (a vector the same length as `Legend`) with the corresponding values of the plotting unit must also be given » :

Argument	Ce qu'il désigne
<code>fill=v</code>	« Colors for filled boxes »
<code>col=v</code>	« Colors in which points or lines will be drawn »
<code>lty=v</code>	« Line styles »
<code>lwd=v</code>	« Line widths »
<code>pch=v</code>	« Plotting characters (character vector) »

DÉFINITION — LES COORDONNÉES UTILISATEUR (§12.2).

« Low-level plotting functions usually require some positioning information ... **Coordinates are given in terms of user coordinates which are defined by the previous high-level graphics command and are chosen based on the supplied data.** »

*« Where `x` and `y` arguments are required, **it is also sufficient to supply a single argument being a list with elements named `x` and `y`**. Similarly a matrix with two columns is also valid **input**. In this way functions such as `locator()` may be used to specify positions on a plot interactively. »*

C'est la raison pour laquelle il faut un tracé de haut niveau avant tout bas niveau : sans lui, le système de coordonnées n'existe pas.

2.1 Annotation mathématique et polices de Hershey

MÉTHODE (§12.2.1).

« *This can be achieved in R by specifying an expression rather than a character string in any one of `text`, `mtext`, `axis`, or `title`. For example, the following code draws the formula for the Binomial probability function* » :

```
text(x, y, expression(paste(bgroup("(", atop(n, x), ")"), p^x, q^{n-x})))
```

Le résultat est $\binom{n}{x} p^x q^{n-x}$. On reconnaît les briques : `bgroup("(", atop(n, x), ")")` empile n sur x entre parenthèses, `p^x` et `q^{n-x}` posent les exposants.

« More information ... can be obtained from within R using `help(plotmath)`, `example(plotmath)`, `demo(plotmath)` . »

COMPLÉMENT (§12.2.2).

« *It is possible to specify **Hershey vector fonts** for rendering text when using the `text` and `contour` functions. There are three reasons for using the Hershey fonts* » :

1. « *better output, especially on a computer screen, for rotated and/or small text* » ;
2. « *certain symbols that may not be available in the standard fonts — in particular, **zodiac signs, cartographic symbols and astronomical symbols*** » ;
3. « *Cyrillic and Japanese (Kana and Kanji) characters* ».

Documentation : `help(Hershey)`, `demo(Hershey)`, `help(Japanese)`, `demo(Japanese)` .

● Concept 3 — Interagir avec un graphique

DÉFINITION (§12.3).

« `locator(n, type)` — **Waits for the user to select locations on the current plot using the left mouse button.** This continues until `n` (default 512) points have been selected, or **another mouse button is pressed**. ... `locator()` **returns the locations of the points selected as a list with two components `x` and `y`.** »

« `locator()` **is usually called with no arguments.** It is particularly useful for **interactively selecting positions for graphic elements such as legends or labels when it is difficult to calculate in advance where the graphic should be placed.** »

```
text(locator(1), "Outlier", adj = 0)
```

« `locator()` **will be ignored if the current device, such as `postscript`, does not support interactive pointing.** »

DÉFINITION (§12.3).

« `identify(x, y, labels)` — **Allow the user to highlight any of the points defined by `x` and `y` ... by plotting the corresponding component of `labels` nearby (or the index number of the point if `labels` is absent).** Returns the indices of the selected points when another button is pressed. »

« **The `identify()` function performs no plotting itself, but simply allows the user to move the mouse pointer and click ... If there is a point near the mouse pointer it will be marked with its index number ... or disable marking altogether with the `plot = FALSE` argument.** When the process is terminated, `identify()` **returns the indices of the selected points ; you can use these indices to extract the selected points from the original vectors.** »

La différence entre les deux : `locator()` rend **des positions** (où l'on a cliqué), `identify()` rend **des indices** (quelles observations). « Sometimes we want to identify **particular points** on a plot, **rather than their positions.** »

● Concept 4 — Les paramètres graphiques

Cadrage (§12.4). « *R's defaults do not always produce exactly that which is required. You can, however, customize almost every aspect of the display using graphics parameters. ... Every graphics parameter has a name (such as `col`) and a value.* »

« *A separate list of graphics parameters is maintained for each active device, and each device has a default set when initialized. Graphics parameters can be set in two ways : either permanently, affecting all graphics functions which access the current device ; or temporarily, affecting only a single graphics function call.* »

4.1 `par()` — et son piège

Appel (§12.4.1)	Effet
<code>par()</code>	« <i>Without arguments, returns a list of all graphics parameters and their values for the current device</i> »
<code>par(c("col", "lty"))</code>	« <i>With a character vector argument, returns only the named graphics parameters</i> »
<code>par(col=4, lty=2)</code>	« <i>sets the values ... and returns the original values of the parameters as a list</i> »

Le piège central du chapitre (§12.4.1). « *Note that calls to `par()` always affect the global values of graphics parameters, even when `par()` is called from within a function. This is often undesirable behavior — usually we want to set some graphics parameters, do some plotting, and then restore the original values so as not to affect the user's R session.* »

Une fonction qui appelle `par()` modifie la session de l'utilisateur. C'est l'unique construction du langage, avec les environnements (fiche 303), qui échappe à la sémantique de copie.

MÉTHODE — L'IDIOME DE RESTAURATION (§12.4.1).

« *You can restore the initial values by saving the result of `par()` when making changes, and restoring the initial values when plotting is complete.* »

```
oldpar <- par(col = 4, lty = 2)
# ... commandes de trace ...
par(oldpar)
```

« *To save and restore all settable graphical parameters use* » :

```
oldpar <- par(no.readonly = TRUE)
# ... commandes de trace ...
par(oldpar)
```

(note 1 : « *Some graphics parameters such as the size of the current device are for information only.* »)

C'est l'idiome vu à l'annexe A (fiche 300) et avant un Q-Q (fiche 312). `no.readonly = TRUE` exclut les paramètres en lecture seule, seuls les autres pouvant être repassés à `par()`.

4.2 Le réglage temporaire

Règle (§12.4.2). « *Graphics parameters may also be passed to (almost) any graphics function as named arguments. This has the same effect as passing the arguments to `par()`, except that the changes only last for the duration of the function call.* »

```
plot(x, y, pch = "+") # sans changer le caractère par défaut des traces suivantes
```

« *Unfortunately, this is not implemented entirely consistently and it is sometimes necessary to set and reset graphics parameters using `par()`.* »

4.3 Les éléments graphiques

NOTATION (§12.5.1).

« *R plots are made up of points, lines, text and polygons (filled regions).* »

Paramètre	Effet
<code>pch</code>	« Character to be used for plotting points. » Un caractère, ou un entier de 0 à 25 : « a specialized plotting symbol ». « <i>Those from 21 to 25 may appear to duplicate earlier symbols, but can be coloured in different ways</i> ». Aussi 32:255 pour un caractère de la police courante
<code>lty</code>	« Line types. ... line type 1 is always a solid line, line type 0 is always invisible, and line types 2 and onwards are dotted or dashed » — « <i>Alternative line styles are not supported on all graphics devices</i> »
<code>lwd</code>	« Line widths ... in multiples of the "standard" line width. Affects axis lines as well »
<code>col</code>	« Colors ... A number from the current palette (see <code>?palette</code>) or a named colour »
<code>col.axis</code> · <code>col.lab</code> · <code>col.main</code> · <code>col.sub</code>	la couleur « <i>for axis annotation, x and y labels, main and sub-titles, respectively</i> »
<code>font</code>	« 1 plain text, 2 bold face, 3 italic, 4 bold italic and 5 a symbol font (which include Greek letters) »
<code>font.axis</code> · <code>font.lab</code> · <code>font.main</code> · <code>font.sub</code>	idem, par composante
<code>adj</code>	« Justification of text relative to the plotting position. 0 left, 1 right, 0.5 centered ... the actual value is the proportion of text that appears to the left of the plotting position, so a value of -0.1 leaves a gap of 10 % of the text width »
<code>cex</code>	« Character expansion. ... the desired size of text characters (including plotting characters) relative to the default »
<code>cex.axis</code> · <code>cex.lab</code> · <code>cex.main</code> · <code>cex.sub</code>	idem, par composante

MÉTHODE — VOIR LES 26 SYMBOLES (§12.5.1).

« To see what the symbols are, use the command »

```
legend(locator(1), as.character(0:25), pch = 0:25)
```

Une ligne qui combine **trois** choses de ce chapitre : `locator(1)` pour choisir l'emplacement à la souris, `legend()` pour la boîte, et le fait que `pch` accepte **un vecteur**.

4.4 Axes et graduations

Paramètre (§12.5.2)	Effet
<code>lab=c(5, 7, 12)</code>	« The first two numbers are the desired number of tick intervals on x and y. The third number is the desired length of axis labels, in characters (including the decimal point). » « Choosing a too-small value ... may result in all tick labels being rounded to the same number! »
<code>las=1</code>	« Orientation of axis labels. 0 always parallel to axis, 1 always horizontal, 2 always perpendicular »
<code>mgp=c(3, 1, 0)</code>	« Positions of axis components ... the axis label , the tick labels , the axis line ... Positive numbers measure outside the plot region, negative numbers inside. »
<code>tck=0.01</code>	« Length of tick marks, as a fraction of the size of the plotting region. ... A value of 1 gives grid lines. Negative values give tick marks outside. Use <code>tck=0.01</code> and <code>mgp=c(1, -1.5, 0)</code> for internal tick marks. »
<code>xaxs</code> · <code>yaxs</code>	« styles "i" (internal) and "r" (the default) ... style "r" leaves a small amount of space at the edges »

« Note that `axes` **is not a graphics parameter but an argument to a few plot methods** : see `xaxt` and `yaxt` . » (§12.5)

4.5 Marges et figures multiples

DÉFINITION (§12.5.3).

« **A single plot in R is known as a figure and comprises a plot region surrounded by margins (possibly containing axis labels, titles, etc.) and (usually) bounded by the axes themselves.** »

Paramètre	Effet
<code>mai=c(1, 0.5, 0.5, 0)</code>	« Widths of the bottom, left, top and right margins, respectively, measured in inches »
<code>mar=c(4, 2, 2, 1)</code>	« Similar to <code>mai</code> , except the measurement unit is text lines »

« `mar` and `mai` are equivalent in the sense that setting one changes the value of the other. The default values chosen for this parameter are often too large ; the right-hand margin is rarely needed, and neither is the top margin if no title is being used. ... the default is chosen without regard to the size of the device surface : for example, using `postscript()` with `height=4` will result in a plot which is about 50 % margin unless `mar` or `mai` are set explicitly. »

DÉFINITION (§12.5.4).

« **R allows you to create an n by m array of figures on a single page. Each figure has its own margins, and the array of figures is optionally surrounded by an outer margin.** »

Paramètre	Effet
<pre>mfcol=c(3, 2) · mfrow=c(2, 4)</pre>	« <i>The first value is the number of rows ; the second is the number of columns. The only difference ... is that setting <code>mfcol</code> causes figures to be filled by column ; <code>mfrow</code> fills by rows.</i> »
<pre>mfg=c(2, 2, 3, 2)</pre>	« <i>Position of the current figure ... the first two are the row and column of the current figure, the last two the number of rows and columns ... Set this parameter to jump between figures in the array.</i> »
<pre>fig=c(4, 9, 1, 4)/10</pre>	« <i>Position of the current figure on the page ... left, right, bottom and top edges ... as a percentage of the page measured from the bottom left corner. If you want to add a figure to a current page, use <code>new=TRUE</code> as well (unlike S).</i> »
<pre>oma=c(2, 0, 3, 0) · omi=c(0, 0, 0.8, 0)</pre>	« <i>Size of outer margins. Like <code>mar</code> and <code>mai</code>, the first in text lines and the second in inches, starting with the bottom margin and working clockwise.</i> »

La réduction automatique de taille (§12.5.4). « *Setting either of these can reduce the base size of symbols and text (controlled by `par("cex")` and the pointsize of the device). In a layout with exactly two rows and columns the base size is reduced by a factor of 0.83 ; if there are three or more of either rows or columns, the reduction factor is 0.66.* »

Deux seuils, deux facteurs. Passer de 2×2 à 2×3 fait chuter la taille de 0,83 à **0,66** — d'un coup, pas progressivement.

« *Outer margins are particularly useful for page-wise titles. Text can be added to the outer margins with the `mtext()` function with argument `outer=TRUE`. There are no outer margins by default, however, so you must create them explicitly using `oma` or `omi`.* »

« *More complicated arrangements of multiple figures can be produced by the `split.screen()` and `layout()` functions, as well as by the `grid` and `Lattice` packages.* »

Énoncé. Écrire une fonction qui trace un nuage étiqueté, avec des axes personnalisés et une légende, sans polluer la session de l'utilisateur.

Étape 1 — protéger la session. C'est **la première ligne**, avant tout tracé : « calls to `par()` **always affect the global values ... even when `par()` is called from within a function** » (§12.4.1).

```
nuage <- function(x, y, noms) {
  oldpar <- par(no.readonly = TRUE)
  on.exit(par(oldpar))
  ...
}
```

on.exit() **garantit la restauration même en cas d'erreur** — le `par(oldpar)` du cours, placé en fin de fonction, serait sauté si une commande intermédiaire échouait. (Cette précision est un enrichissement : le §12.4.1 donne l'idiome, pas la protection contre l'erreur.)

Étape 2 — préparer le terrain sans rien tracer. `type="n"` : « **No plotting at all. However axes are still drawn (by default) and the coordinate system is set up according to the data.** » Et `axes=FALSE` pour reprendre la main dessus.

```
par(mar = c(4, 4, 2, 1))      # marges resserrées : le défaut est « often too large
plot(x, y, type = "n", axes = FALSE, xlab = "", ylab = "")
```

Étape 3 — poser les axes soi-même. `axis(side, ...)` — « **1 to 4, counting clockwise from the bottom** ».

```
axis(1); axis(2, las = 1)      # las = 1 : étiquettes TOUJOURS horizontales
title(main = "Nuage étiqueté", xlab = "x", ylab = "y")
```

Étape 4 — le contenu. L'idiome du §12.2 : **remplacer les points par les étiquettes**.

```
text(x, y, noms, cex = 0.8, adj = 0.5)
```

`adj = 0.5` **centre horizontalement** sur la position ; `cex = 0.8` réduit le corps à 80 % du défaut.

Étape 5 — repères et légende.

```
abline(h = mean(y), lty = 2)   # lty = 2 : pointille
legend("topleft", "moyenne de y", lty = 2, bty = "n")
```

`legend()` exige « **at least one other argument** » ... *with the corresponding values of the plotting unit* » — ici `lty`, sans quoi la boîte n'aurait aucun symbole à montrer.

Étape 6 — vérifier l'ordre. **Haut niveau d'abord**, bas niveau ensuite : « *High-level plotting commands always start a new plot, erasing the current plot if necessary* ». Un `plot()` glissé au milieu effacerait tout le travail.

Étape 7 — pourquoi le système de coordonnées existe. `abline(h = mean(y))` et `text(x, y, ...)` s'expriment en **coordonnées utilisateur**, « *defined by the previous high-level graphics command and chosen based on the supplied data* ». Sans l'étape 2, aucune de ces commandes n'aurait de sens.

Étape 8 — le bilan. La fonction ne laisse **aucune trace** dans la session : `on.exit(par(oldpar))` restaure tout, y compris les marges. C'est exactement ce que le cours demande — « *so as not to affect the user's R session* ».

● Concept 5 — Les pilotes de périphérique

DÉFINITION (§12.6).

« *R can generate graphics ... on almost any type of display or printing device. Before this can begin, however, R needs to be informed what type of device it is dealing with. This is done by starting a device driver. The purpose of a device driver is to convert graphical instructions from R ("draw a line," for example) into a form that the particular device can understand.* »

« *There is one such function for every device driver : type `help(Devices)` for a list of them all.* »

Pilote	Usage
<code>x11()</code>	« For use with the X11 window system on Unix-alikes »
<code>windows()</code>	« For use on Windows »
<code>quartz()</code>	« For use on macOS »
<code>postscript()</code>	« For printing on PostScript printers, or creating PostScript graphics files »
<code>pdf()</code>	« Produces a PDF file »
<code>png()</code>	« a bitmap PNG file. (Not always available) »
<code>jpeg()</code>	« a bitmap JPEG file, best used for image plots. (Not always available) »

Ne jamais oublier de fermer (§12.6). *« When you have finished with a device, be sure to terminate the device driver by issuing the command `dev.off()` . This ensures that the device finishes cleanly ; for example in the case of **hardcopy devices this ensures that every page is completed and has been sent to the printer. (This will happen automatically at the normal end of a session.) »***

Un `pdf("f.pdf")` sans `dev.off()` laisse un fichier incomplet, souvent illisible. C'est la panne la plus fréquente de la production de graphiques en script.

5.1 PostScript pour un document

MÉTHODE (§12.6.1).

« **The plot will be in landscape orientation unless the `horizontal=FALSE` argument is given, and you can control the size of the graphic with the `width` and `height` arguments.** »

```
postscript("file.ps", horizontal = FALSE, height = 5, pointsize = 10)
```

« *It is important to note that if the file named in the command already exists, it will be overwritten. This is the case even if the file was only created earlier in the same R session.* »

Règle — l'EPS (§12.6.1). « *Many usages ... will be to incorporate the figure in another document. This works best when encapsulated PostScript is produced : R always produces conformant output, but only marks the output as such when the `onefile=FALSE` argument is supplied. This unusual notation stems from S-compatibility : it really means that the output will be a single page (which is part of the EPSF specification).* »

```
postscript("plot1.eps", horizontal = FALSE, onefile = FALSE,  
           height = 8, width = 6, pointsize = 10)
```

`onefile=FALSE` **ne désactive pas un fichier unique** — il **déclare** que la sortie tient en **une page**. Le nom dit le contraire de ce qu'on croit ; le cours l'appelle « *this unusual notation* ».

5.2 Plusieurs périphériques

Cadrage (§12.6.2). « *Of course only one graphics device can accept graphics commands at any one time, and this is known as the current device. When multiple devices are open, they form a numbered sequence with names giving the kind of device at any position.* »

Commande	Effet
tout appel de pilote	« Each new call to a device driver function opens a new graphics device, thus extending by one the device list. This device becomes the current device. »
<code>dev.list()</code>	« <i>Returns the number and name of all active devices. The device at position 1 on the list is always the null device which does not accept graphics commands at all.</i> »
<code>dev.next()</code> · <code>dev.prev()</code>	le périphérique suivant ou précédent
<code>dev.set(which=k)</code>	« change the current graphics device to the one at position <code>k</code> »
<code>dev.off(k)</code>	« Terminate the graphics device at point <code>k</code> ... this will either print the file immediately or correctly complete the file for later printing »
<code>dev.copy(device, ..., which=k)</code>	« Make a copy of the device <code>k</code> »
<code>dev.print(device, ..., which=k)</code>	« <i>similar, but the copied device is immediately closed, so that end actions, such as printing hardcopies, are immediately performed</i> »
<code>graphics.off()</code>	« Terminate all graphics devices on the list, except the null device. »

● Concept 6 — Graphiques dynamiques

Règle (§12.7). *« R does not have builtin capabilities for dynamic or interactive graphics, e.g. rotating point clouds or "brushing" (interactively highlighting) points. However, extensive dynamic graphics facilities are available in the system **GGobi** by Swayne, Cook and Buja ... accessed from R via the package `rggobi` . »*

« Also, **package `rgl` provides ways to interact with 3D plots, for example of surfaces.** »

Comment reconnaître le type d'exercice

Le signal dans l'énoncé	Ce qu'il faut mobiliser
« tracer y contre x »	<code>plot(x, y)</code> — haut niveau
« des boîtes à moustaches par groupe »	<code>plot(f, y)</code> avec <code>f</code> facteur
« chaque variable contre toutes les autres »	<code>pairs(X)</code>
« y contre x, selon z »	<code>coplot(a ~ b c)</code>
« changer le contenu de chaque case »	l'argument <code>panel=</code>
« ajouter une droite de régression »	<code>abline(lm.obj)</code>
« une ligne horizontale au niveau de la moyenne »	<code>abline(h = mean(y))</code>
« étiqueter les points par leur nom »	<code>plot(x, y, type="n"); text(x, y, names)</code>
« mes axes ne me conviennent pas »	<code>axes=FALSE</code> puis <code>axis()</code>
« superposer sans effacer »	<code>add=TRUE</code> — ou une fonction de bas niveau
« échelle logarithmique »	<code>log="x"</code> , <code>"y"</code> , <code>"xy"</code>
« des bâtons verticaux »	<code>type="h"</code>
« préparer le cadre sans rien tracer »	<code>type="n"</code>
« une formule mathématique sur le graphique »	<code>expression()</code> — voir <code>plotmath</code>
« du texte pivoté ou du cyrillique »	les polices de Hershey
« cliquer pour placer une légende »	<code>locator(1)</code>
« cliquer pour désigner des observations »	<code>identify()</code> — rend des indices
« plusieurs graphiques sur une page »	<code>mfrow</code> ou <code>mfcol</code>
« un titre commun aux quatre graphiques »	<code>oma</code> + <code>mtext(outer=TRUE)</code>
« mes marges sont trop grandes »	<code>mar</code> ou <code>mai</code> — « <i>often too large</i> »
« mon fichier PDF est illisible »	<code>dev.off()</code> oublié

Le signal dans l'énoncé	Ce qu'il faut mobiliser
« une figure à inclure dans un document »	<code>onefile=FALSE</code>
« ma fonction change les réglages de l'utilisateur »	<code>oldpar <- par(no.readonly=TRUE)</code>
« faire tourner un nuage en 3D »	pas en base R — <code>rgl</code> , <code>rggobi</code>

Comment résoudre ce type d'exercice

Protocole « composer un graphique » — 5 étapes.

1. **Un seul appel de haut niveau**, en premier — il **efface** tout ce qui précède.
2. Si le tracé doit être entièrement maîtrisé : `type="n"` et `axes=FALSE` .
3. **Ajouter par le bas niveau** : `points` , `lines` , `text` , `abline` , `polygon` , `axis` , `title` , `legend` .
4. Toutes les positions sont en **coordonnées utilisateur**, fixées par l'étape 1.
5. **Une légende exige au moins un vecteur** de style (`lty` , `pch` , `col` , `fill` , `lwd`).

Protocole « régler les paramètres sans nuire » — 4 étapes.

1. Réglage **d'un seul tracé** → passer le paramètre **en argument** de la fonction graphique.
2. Réglage **durable** → `par()` , mais **il est global, même depuis une fonction**.
3. **Sauvegarder** : `oldpar <- par(no.readonly = TRUE)` .
4. **Restaurer** : `par(oldpar)` — de préférence par `on.exit()` .

Protocole « produire un fichier graphique » — 4 étapes.

1. **Ouvrir le pilote** : `pdf("f.pdf")` , `png("f.png")` , `postscript("f.eps", onefile = FALSE, horizontal = FALSE)` .
2. **Tracer** — le fichier **est écrasé s'il existe**, même créé dans la même session.
3. `dev.off()` — sans quoi le fichier est **incomplet**.
4. Plusieurs périphériques ouverts : `dev.list()` , `dev.set(k)` , `graphics.off()` pour tout fermer.

● Common mistakes

Erreur	Correction
Enchaîner deux fonctions de haut niveau	la seconde efface la première
Utiliser <code>lines()</code> avant tout <code>plot()</code>	aucun système de coordonnées n'existe
Croire <code>plot()</code> toujours identique	il est générique — le type dépend de la classe
Attendre exactement <code>nclass</code> classes	c'est « a recommendation » — <code>breaks</code> est exact
Oublier <code>probability=TRUE</code> avant une densité	les barres sont en effectifs
Appeler <code>par()</code> dans une fonction sans restaurer	« always affect the global values »
Restaurer par <code>par(oldpar)</code> en fin de corps	une erreur intermédiaire saute la ligne → <code>on.exit()</code>
Croire que tous les paramètres se repassent à <code>par()</code>	<code>no.readonly = TRUE</code> exclut ceux en lecture seule
Croire les réglages temporaires toujours possibles	« not implemented entirely consistently »
Passer une légende sans vecteur de style	il en faut au moins un
Choisir un troisième nombre trop petit dans <code>lab</code>	« all tick labels being rounded to the same number »
Garder les marges par défaut sur un petit périphérique	« about 50 % margin » — régler <code>mar / mai</code>
Attendre une marge extérieure par défaut	« There are no outer margins by default »
Croire la réduction de taille progressive	0,83 puis 0,66 , par seuils
Ajouter une figure sans <code>new=TRUE</code>	requis avec <code>fig=</code> , « unlike S »
Oublier <code>dev.off()</code>	fichier incomplet
Croire écrire dans un fichier existant sans risque	« it will be overwritten »
Croire <code>onefile=FALSE</code> interdire le fichier unique	il signifie une seule page — notation « <i>unusual</i> »
Fermer le périphérique 1	c'est le périphérique nul
Attendre du 3D interactif en base R	absent — <code>rgl</code> , <code>rggobi</code>

Ultimate Review

Trois groupes. Haut niveau — « *create a new plot ... always start a new plot, erasing the current plot if necessary* » · **bas niveau** — « *add more information to an existing plot* » · **interactif** — à la souris. Plus une liste de paramètres graphiques.

Périmètre. « *This manual only describes what are known as "base" graphics* » · `grid` « *more powerful but harder to use* » · `lattice` pour les tracés multi-panneaux « *akin to ... Trellis* ».

Ouverture. `x11()` (UNIX), `windows()`, `quartz()` (macOS) · `dev.new()` ouvre un nouveau périphérique.

`plot()` est générique. Vecteurs → nuage · un vecteur → **valeurs contre index** · série temporelle → **chronogramme** · complexe → **imaginaire contre réel** · facteur → **barres** · facteur + numérique → **boîtes à moustaches** · data frame → **tracés distributionnels** · `y ~ expr` → `y` contre **chaque objet nommé**.

Multivarié. `pairs(X)` — $n(n-1)$ tracés, « *plot scales constant over the rows and columns* » · `coplot(a ~ b | c)` et `c + d` ; `c` facteur → **par niveau**, `c` numérique → **par intervalle de conditionnement**, réglé par `given.values=` et `co.intervals()` · `panel=` pour changer le tracé de chaque case, défaut `points()`, exemple `panel.smooth()`.

Autres tracés. `qqnorm` / `qqline` (« *through the distribution and data quartiles* ») / `qqplot` · `hist` avec `nclass` (**recommandation**) ou `breaks` (**exact**), et `probability=TRUE` · `dotchart` · `image`, `contour`, `persp`.

Arguments de haut niveau. `add=TRUE` (« *some functions only* ») · `axes=FALSE` · `log=` · `xlab` `ylab` `main` `sub` · `type=` : `p` `l` `b` `o` `h` `s` `S` `n` — ce dernier « *sets up the coordinate system ... ideal for ... subsequent low-level graphics* ».

Bas niveau. `points` (défaut "p") · `lines` (défaut "l") · `text` (défaut des étiquettes : `1:length(x)`) · `abline(a,b)`, `abline(h=)`, `abline(v=)`, `abline(lm.obj)` (« *an intercept and slope, in that order* ») · `polygon` · `legend` (**au moins un vecteur** : `fill`, `col`, `lty`, `lwd`, `pch`) · `title` · `axis(side)` — 1 à 4, dans le sens horaire depuis le bas.

Coordonnées utilisateur. « *defined by the previous high-level graphics command and are chosen based on the supplied data* » · on peut fournir **une liste** `x / y` ou **une matrice** à deux colonnes.

Annotation. `expression()` dans `text`, `mtext`, `axis`, `title` — `help(plotmath)`, `example(plotmath)`, `demo(plotmath)` · **polices de Hershey** : meilleur rendu **en petit et en pivoté**, **symboles** (zodiaque, cartographie, astronomie), **cyrillique et japonais**.

Interaction. `locator(n, type)` — défaut 512, rend **une liste** `x / y`, « *usually called with no arguments* », **ignoré** sur un périphérique non interactif · `identify(x, y, labels)` — « *performs no plotting itself* », marque **l'index** si `labels` absent, `plot=FALSE` pour ne rien marquer, **rend les indices**.

`par()` . Sans argument → **tous les paramètres** · vecteur de caractères → **ceux nommés** · arguments nommés → **règle et rend les anciennes valeurs** · « *calls to `par()` always affect the global values ... even when `par()` is called from within a function* » · idiome : `oldpar <- par(no.readonly = TRUE)` ... `par(oldpar)` · réglage **temporaire** en argument d'une fonction graphique, « *not implemented entirely consistently* ».

Éléments. `pch` — caractère, **0-25**, ou **32:255** ; **21-25 colorables différemment** · `lty` — **1 pleine, 0 invisible**, 2+ pointillés · `lwd` — multiples, **affecte aussi les axes** · `col` — numéro de palette ou **nom** · `font` — 1 normal, 2 gras, 3 italique, 4 gras italique, **5 symboles (grec)** · `adj` — « *the proportion of text that appears to the left* » · `cex` — taille relative · déclinaisons `.axis` `.lab` `.main` `.sub` .

Axes. `lab` (**troisième nombre trop petit** → **étiquettes toutes arrondies pareil**) · `las` (0 parallèle, 1 horizontal, 2 perpendiculaire) · `mgp` (**positif dehors, négatif dedans**) · `tck` (**1 donne des lignes de grille**) · `xaxs` / `yaxs` ("r" laisse de l'espace) · `axes` n'est pas un paramètre graphique.

Marges. `mai` en **pouces**, `mar` en **lignes de texte** — « *equivalent ... setting one changes the other* », et souvent trop grandes par défaut.

Figures multiples. `mfrow` remplit **par lignes**, `mfcol` **par colonnes** — « *the first value is the number of rows* » · `mfg` pour **sauter** d'une figure à l'autre · `fig` en **pourcentage de page depuis le coin inférieur gauche**, avec `new=TRUE` « *unlike S* » · `oma` / `omi` , **du bas dans le sens horaire, inexistantes par défaut** · texte par `mtext(outer=TRUE)` · réduction de taille **0,83** en **2 × 2**, **0,66** au-delà · plus complexe : `split.screen()` , `layout()` , `grid` , `lattice` .

Périphériques. `help(Devices)` · `postscript` , `pdf` , `png` , `jpeg` , `tiff` , `bitmap` · `dev.off()` **obligatoire** · **le périphérique 1 est le périphérique nul** · `dev.list` , `dev.next` , `dev.prev` , `dev.set(which=k)` , `dev.copy` , `dev.print` (« *the copied device is immediately closed* »), `graphics.off()` .

PostScript. `horizontal=FALSE` pour le portrait · **le fichier existant est écrasé** · `onefile=FALSE` marque la sortie comme **encapsulée** — « *it really means that the output will be a single page* ».

Dynamique. « *R does not have builtin capabilities for dynamic or interactive graphics* » — **GGobi** via `rggobi` , et `rgl` pour la 3D.

Active Recall

« *Plotting commands are divided into three basic groups* » (§12) :

1. **Haut niveau** — « *create a new plot on the graphics device, possibly with axes, labels, titles* » ;
2. **Bas niveau** — « *add more information to an existing plot, such as extra points, lines and labels* » ;
3. **Interactives** — « *allow you interactively add information to, or extract information from, an existing plot, using a pointing device such as a mouse* ».

C'est le haut niveau qui efface : « *High-level plotting commands always start a new plot, erasing the current plot if necessary.* » (§12.1)

Savoir dans quel groupe on est suffit à prévoir le comportement.

« *This is a generic function : the type of plot produced is dependent on the type or class of the first argument.* » (§12.1.1)

C'est le dispatch S3 de la fiche 311 :

Premier argument	Résultat
deux vecteurs	nuage de points
un vecteur numérique	valeurs contre leur index
une série temporelle	un chronogramme
un vecteur complexe	imaginaire contre réel
un facteur	un diagramme en barres
facteur + vecteur numérique	des boîtes à moustaches par niveau
un data frame	des tracés distributionnels

Cela explique deux surprises de l'annexe A (fiche 300) : `plot(Expt, Speed)` donnant des boîtes, et `plot(z, type="l")` traçant un cercle.

« `type="n"` — No plotting at all. However axes are still drawn (by default) and the coordinate system is set up according to the data. Ideal for creating plots with subsequent low-level graphics functions. » (§12.1.4)

Il prépare le terrain sans rien y mettre : cadre, échelles, axes — puis on ajoute exactement ce qu'on veut.

L'usage canonique est donné au §12.2 :

```
plot(x, y, type = "n"); text(x, y, names)
```

« **The graphics parameter `type="n"` suppresses the points but sets up the axes, and the `text()` function supplies special characters ... for the points.** » — **un nuage où chaque observation porte son nom** au lieu d'un symbole.

*« **When three or four variables are involved a coplot may be more enlightening.**

`coplot(a ~ b | c)` produces a number of scatterplots of `a` against `b` for given values of `c`. »* (§12.1.2)

- **`c` facteur** : « this simply means that `a` is plotted against `b` for every level of `c` » ;
- **`c` numérique** : « it is divided into a number of conditioning intervals and for each interval `a` is plotted against `b` for values of `c` within the interval ».

« **The number and position of intervals can be controlled with `given.values=`** — the function `co.intervals()` is useful for selecting intervals. »

Et `coplot(a ~ b | c + d)` produit des nuages « **for every joint conditioning interval of `c` and `d`** ».

« **The `coplot()` and `pairs()` function both take an argument `panel=` which can be used to customize the type of plot which appears in each panel. The default is `points()` ... by supplying some other low-level graphics function of two vectors `x` and `y` as the value of `panel=` you can produce any type of plot you wish. An example panel function useful for coplots is `panel.smooth()`.** » (§12.1.2)

C'est une fonction passée en argument — le mécanisme de la fiche 309, où « *functions ... can be **passed as arguments to functions*** ». Exactement comme `outer(x, y, f)` de la fiche 305.

La contrainte est **la signature** : `panel=` doit recevoir une fonction « *of two vectors `x` and `y`* ».

« **A sensible number of classes is usually chosen, but a recommendation can be given with the `nclass=` argument. Alternatively, the breakpoints can be specified exactly with the `breaks=` argument.** » (§12.1.3)

Le cours choisit ses mots : `nclass` est « *a recommendation* », `breaks` est « *specified **exactly*** ». Un `nclass = 20` peut très bien donner 18 ou 25 classes — R ajuste pour obtenir des bornes « rondes ».

Pour un contrôle exact, il faut passer **le vecteur des bornes**, comme au §8.2 (fiche 312) :
`hist(eruptions, seq(1.6, 5.2, 0.2), prob = TRUE)` .

Et : « **If the `probability=TRUE` argument is given, the bars represent relative frequencies divided by bin width instead of counts.** »

« *Low-level plotting functions usually require some positioning information ... **Coordinates are given in terms of user coordinates which are defined by the previous high-level graphics command and are chosen based on the supplied data.*** » (§12.2)

La contrainte est un ordre obligatoire : il faut **une commande de haut niveau d'abord**, sinon le système de coordonnées **n'existe pas** et aucune fonction de bas niveau n'a de sens.

« *Where `x` and `y` arguments are required, **it is also sufficient to supply a single argument being a list with elements named `x` and `y`** ... Similarly a matrix with two columns is also valid input.* »

C'est ce qui permet `text(locator(1), "Outlier", adj = 0)` : `locator()` rend « **a list with two components `x` and `y`** », directement utilisable.

« Sometimes we want to identify **particular points** on a plot, **rather than their positions**. »
 (§12.3)

	<code>locator()</code>	<code>identify()</code>
Ce qu'il rend	les positions cliquées — liste <code>x / y</code>	les indices des points sélectionnés
Trace-t-il ?	selon <code>type</code> (défaut : rien)	« performs no plotting itself », mais marque le point
Nombre par défaut	512	—
Usage typique	placer une légende, une étiquette	désigner des observations

« `Locator()` is usually called with no arguments. » Et « will be ignored if the current device, such as `postscript`, does not support interactive pointing ».

Pour `identify()` : « or **disable marking altogether with the `plot = FALSE` argument ... you can use these indices to extract the selected points from the original vectors** ».

« **Note that calls to `par()` always affect the global values of graphics parameters, even when `par()` is called from within a function. This is often undesirable behavior** — usually we want to set some graphics parameters, do some plotting, and then **restore the original values so as not to affect the user's R session.** » (§12.4.1)

C'est une exception à la sémantique de copie de R (fiche 309 : « assignments ... are local and temporary »), au même titre que les environnements (fiche 303).

L'idiome :

```
oldpar <- par(no.readonly = TRUE)
# ... commandes de trace ...
par(oldpar)
```

`par()` « *returns the original values of the parameters as a list* » — c'est ce qui permet de les ranger. Et `no.readonly = TRUE` exclut les paramètres « *for information only* », seuls les autres pouvant être repassés.

« *Graphics parameters may also be passed to (almost) any graphics function as named arguments. This has the same effect as passing the arguments to `par()`, except that the changes only last for the duration of the function call.* » (§12.4.2)

```
plot(x, y, pch = "+") # sans changer le default des traces suivants
```

La réserve : « *Unfortunately, this is not implemented entirely consistently and it is sometimes necessary to set and reset graphics parameters using `par()`.* »

Autrement dit : **essayer d'abord en argument**, et se rabattre sur l'idiome `oldpar` si le paramètre n'est pas accepté — ce qui arrive, sans que le cours donne la liste des exceptions.

« *Adds an axis to the current plot on the side given by the first argument (1 to 4, counting clockwise from the bottom). ... Useful for adding custom axes after calling `plot()` with the `axes=FALSE` argument.* » (§12.2)

1 = bas, 2 = gauche, 3 = haut, 4 = droite — dans le sens horaire depuis le bas.

C'est le même ordre que `mar` et `mai` : « *Widths of the bottom, left, top and right margins, respectively* » (§12.5.3), et que `oma` / `omi` : « *starting with the bottom margin and working clockwise* ».

Un seul ordre à retenir pour tout le chapitre.

Le patron complet : `plot(..., axes = FALSE)` puis `axis(1)`, `axis(2)`, puis `title()`.

« *The first value is the number of rows ; the second is the number of columns. The only difference between these two parameters is that setting `mfcol` causes figures to be filled by column ; `mfrow` fills by rows.* » (§12.5.4)

La taille change automatiquement : « *Setting either of these can reduce the base size of symbols and text ... In a layout with exactly two rows and columns the base size is reduced by a factor of 0.83 : if there are three or more of either rows or columns, the reduction factor is 0.66.* »

Deux seuils, pas une progression continue. Passer de 2×2 à 2×3 fait chuter le facteur de **0,83 à 0,66** d'un coup.

Et « *when multiple figures are in use the margins are reduced, however this may not be enough when many figures share the same page* » (§12.5.3).

« *The array of figures is optionally surrounded by an outer margin* » (§12.5.4). Les paramètres sont `oma` (lignes de texte) et `omi` (pouces), « *starting with the bottom margin and working clockwise* ».

« *Outer margins are particularly useful for page-wise titles, etc. Text can be added to the outer margins with the `mtext()` function with argument `outer=TRUE`.* »

Le piège : « *There are no outer margins by default, however, so you must create them explicitly using `oma` or `omi`.* »

Un `mtext("Titre", outer = TRUE)` sans `oma` préalable **n'affiche rien** — il n'y a nulle part où écrire. Aucun message n'est émis.

*« **When you have finished with a device, be sure to terminate the device driver by issuing the command `dev.off()`.** This ensures that the device finishes cleanly ; for example in the case of **hardcopy devices this ensures that every page is completed and has been sent to the printer.** (This will happen automatically at the normal end of a session.) »* (§12.6)

Le fichier reste incomplet — un PDF sans sa table des objets, un PostScript sans sa page finale — et souvent **illisible**. C'est la panne la plus fréquente de la production de graphiques en script, d'autant qu'**aucune erreur n'est levée**.

En session interactive, la fermeture est automatique **à la fin normale** de la session ; en batch (fiche 300), un plantage l'empêche.

Pour tout fermer : `graphics.off()` — « *Terminate all graphics devices on the list, **except the null device*** ».

« *Many usages of PostScript output will be to **incorporate the figure in another document**. This works best when encapsulated PostScript is produced : R always produces conformant output, but only marks the output as such when the `onefile=FALSE` argument is supplied. This unusual notation stems from S-compatibility : it really means that the output will be a single page* (which is part of the EPSF specification). » (§12.6.1)

Le nom dit le contraire de ce qu'il fait : on croit désactiver « un seul fichier », alors qu'on déclare une seule page.

```
postscript("plot1.eps", horizontal = FALSE, onefile = FALSE,  
           height = 8, width = 6, pointsize = 10)
```

Et : « *if the file named in the command already exists, it will be overwritten. This is the case even if the file was only created earlier in the same R session.* »

Flashcards

Question	Réponse
Les trois groupes de commandes ?	Haut niveau, bas niveau, interactives
Que fait une commande de haut niveau ?	Elle crée un tracé complet — et efface le précédent
Une commande de bas niveau ?	Elle ajoute à un tracé existant
Le pilote ouvert au démarrage sous Windows ?	<code>windows()</code>
Sous UNIX ? Sous macOS ?	<code>X11()</code> · <code>quartz()</code>
Ouvrir un nouveau périphérique ?	<code>dev.new()</code>
Quel sous-système ce chapitre décrit-il ?	Les graphiques « base »
Le sous-système alternatif ?	<code>grid</code> — « <i>more powerful but harder to use</i> »
Le paquet des tracés multi-panneaux ?	<code>lattice</code>
Pourquoi <code>plot()</code> varie-t-elle ?	Elle est générique
<code>plot(f)</code> avec <code>f</code> facteur ?	Un diagramme en barres
<code>plot(f, y)</code> ?	Des boîtes à moustaches par niveau
<code>plot(x)</code> avec <code>x</code> complexe ?	Imaginaire contre réel
<code>plot(x)</code> avec <code>x</code> vecteur numérique ?	Les valeurs contre leur index
<code>plot(df)</code> ?	Des tracés distributionnels
Que fait <code>pairs(x)</code> ?	Une matrice de nuages deux à deux
Combien de tracés ?	$n(n - 1)$
Que fait <code>coplot(a ~ b c)</code> ?	<code>a</code> contre <code>b</code> , pour chaque valeur de <code>c</code>
Si <code>c</code> est numérique ?	Il est découpé en intervalles de conditionnement
Comment régler ces intervalles ?	<code>given.values=</code> et <code>co.intervals()</code>
Que fait <code>panel=</code> ?	Il change le tracé de chaque case

Question	Réponse
Son défaut ?	<code>points()</code>
Un exemple utile ?	<code>panel.smooth()</code>
Que fait <code>qqline()</code> ?	Une droite par les quartiles de la loi et des données
<code>nclass</code> dans <code>hist</code> ?	Une recommandation
<code>breaks</code> ?	Les bornes exactes
<code>probability=TRUE</code> ?	Fréquences relatives divisées par la largeur
Que fait <code>dotchart</code> ?	Étiquettes en ordonnée , valeurs en abscisse
Trois tracés de trois variables ?	<code>image</code> , <code>contour</code> , <code>persp</code>
Que fait <code>add=TRUE</code> ?	La fonction agit comme du bas niveau
Que fait <code>axes=FALSE</code> ?	Il supprime les axes — pour les refaire soi-même
<code>type="p"</code> ?	Des points — le défaut
<code>type="l"</code> ?	Des lignes
<code>type="b"</code> contre <code>"o"</code> ?	Points reliés contre points surchargés
<code>type="h"</code> ?	Des bâtons verticaux jusqu'à l'axe zéro
<code>type="s"</code> contre <code>"S"</code> ?	Le haut contre le bas du segment définit le point
<code>type="n"</code> ?	Rien n'est tracé , mais les axes et le repère sont posés
Le défaut de <code>type</code> pour <code>points()</code> ?	<code>"p"</code>
Et pour <code>lines()</code> ?	<code>"l"</code>
Le défaut des étiquettes de <code>text()</code> ?	<code>1:length(x)</code>
<code>abline(a, b)</code> ?	Pente <code>b</code> , ordonnée <code>a</code>
<code>abline(lm.obj)</code> ?	Ordonnée puis pente , dans cet ordre
Que faut-il donner à <code>legend()</code> ?	Au moins un vecteur de style

Question	Réponse
Cinq possibilités ?	<code>fill</code> · <code>col</code> · <code>lty</code> · <code>lwd</code> · <code>pch</code>
Comment sont numérotés les côtés d' <code>axis()</code> ?	1 à 4 , sens horaire depuis le bas
Qui définit les coordonnées utilisateur ?	La commande de haut niveau précédente
Comment écrire une formule sur un graphique ?	Une <code>expression()</code>
Dans quelles fonctions ?	<code>text</code> , <code>mtext</code> , <code>axis</code> , <code>title</code>
Où lire la documentation ?	<code>plotmath</code>
Trois raisons d'utiliser Hershey ?	Petit/pivoté · symboles · cyrillique et japonais
Combien de points <code>locator()</code> attend-il ?	512 par défaut
Que rend-il ?	Une liste <code>x</code> / <code>y</code>
Que rend <code>identify()</code> ?	Les indices des points
Trace-t-il quelque chose ?	Rien lui-même — il marque
Comment désactiver le marquage ?	<code>plot = FALSE</code>
Que rend <code>par()</code> sans argument ?	Tous les paramètres du périphérique courant
Que rend <code>par(col=4)</code> ?	Les anciennes valeurs , en liste
Le piège de <code>par()</code> ?	Il est global , même appelé dans une fonction
L'idiome de sauvegarde ?	<code>oldpar <- par(no.readonly = TRUE)</code>
Que fait <code>no.readonly = TRUE</code> ?	Il exclut les paramètres en lecture seule
Le réglage temporaire ?	Passer le paramètre en argument
Sa limite ?	« <i>not implemented entirely consistently</i> »
Le type de ligne toujours plein ?	1
Le type toujours invisible ?	0
Que fait <code>lwd</code> en plus des lignes ?	Il affecte aussi les axes

Question	Réponse
Que vaut <code>font = 5</code> ?	La police de symboles — lettres grecques
Que signifie <code>adj</code> ?	La proportion de texte à gauche de la position
Que fait <code>cex</code> ?	L' expansion des caractères
Le troisième nombre de <code>lab</code> ?	La longueur des étiquettes , en caractères
Le risque s'il est trop petit ?	Toutes les étiquettes arrondies pareil
Que fait <code>las = 1</code> ?	Étiquettes toujours horizontales
Que donne <code>tck = 1</code> ?	Des lignes de grille
Le style d'axe par défaut ?	<code>"r"</code> — il laisse un peu d'espace
<code>axes</code> est-il un paramètre graphique ?	Non — un argument de méthode
L'unité de <code>mai</code> ?	Les pouces
Celle de <code>mar</code> ?	Les lignes de texte
L'ordre des quatre valeurs ?	Bas, gauche, haut, droite
Les défauts sont-ils bons ?	« <i>often too large</i> »
<code>mfrow</code> remplit comment ?	Par lignes
<code>mfcol</code> ?	Par colonnes
Le premier nombre ?	Le nombre de lignes
Le facteur de réduction en 2×2 ?	0,83
Au-delà ?	0,66
À quoi sert <code>mfg</code> ?	À sauter d'une figure à l'autre
L'unité de <code>fig</code> ?	Un pourcentage de page depuis le coin inférieur gauche
Que faut-il avec <code>fig</code> pour ajouter ?	<code>new=TRUE</code>
Y a-t-il des marges extérieures par défaut ?	Non

Question	Réponse
Comment y écrire ?	<code>mtext(..., outer=TRUE)</code>
Deux fonctions de disposition plus riches ?	<code>split.screen()</code> et <code>layout()</code>
Où lister les pilotes ?	<code>help(Devices)</code>
Que fait <code>dev.off()</code> ?	Il termine proprement le périphérique
Que se passe-t-il si on l'oublie ?	Le fichier est incomplet
Quel périphérique ne prend aucune commande ?	Le périphérique nul , en position 1
Que fait <code>dev.print</code> de plus que <code>dev.copy</code> ?	Il ferme immédiatement la copie
Tout fermer ?	<code>graphics.off()</code>
Comment obtenir du portrait en PostScript ?	<code>horizontal = FALSE</code>
Un fichier existant est-il préservé ?	Non — il est écrasé
Que signifie <code>onefile = FALSE</code> ?	Que la sortie tient en une seule page (EPS)
R fait-il du 3D interactif ?	Non — <code>rgl</code> , <code>rggobi</code>